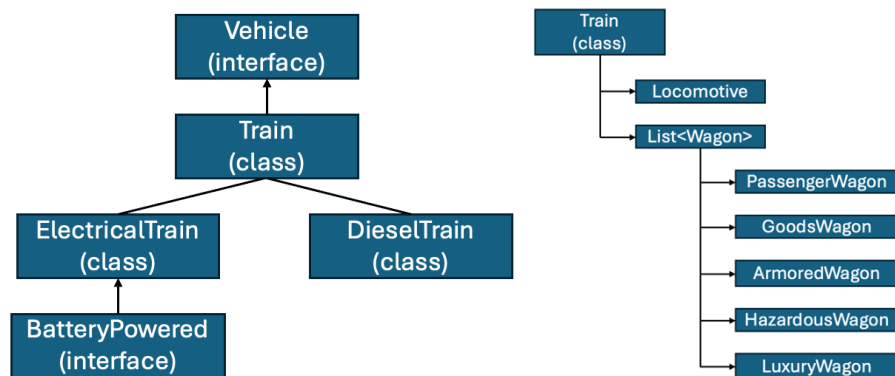


ASSIGNMENTS – OOADI

LECTURE 5

1. A possible class diagram could be as shown here. Also, an overview of suggested components of the Train class is given.



2.

```
public interface Vehicle {
    void start();
    void stop();
    void accelerate(int amount);
    void decelerate(int amount);
    int getSpeed();
}
```



```
public class Train implements Vehicle {
    private Locomotive locomotive;
    private List<Wagon> wagons;
    private int speed;

    @Override
    public void start() { /* common behaviour */ }
    @Override
    public void stop() { speed = 0; }
    @Override
    public void accelerate(int amount) { speed += amount; }
    @Override
    public void decelerate(int amount) { speed -= amount; }
}
```

3. A diesel train would then simply not implement this interface, as it has absolutely no use for it.

```
public interface ElectricTrain {
    void chargeBattery(int kWh);
    int getBatteryLevel();
    boolean needsCharging();
}
```

```
}
```

```
public class BatteryTrain extends Train implements ElectricTrain {  
    private int batteryLevel;  
  
    public BatteryTrain(int initialCharge) {  
        this.batteryLevel = initialCharge;  
    }  
  
    @Override  
    public void chargeBattery(int kWh) { batteryLevel += kWh; }  
    @Override  
    public int getBatteryLevel() { return batteryLevel; }  
    @Override  
    public boolean needsCharging() { return batteryLevel < 20; }  
}
```

-
4. The goal is that any Train object, whether electric or diesel, should behave correctly when used through the Train or Vehicle type.

```
Vehicle v = new BatteryTrain(50);  
v.start(); // fine  
((ElectricTrain) v).chargeBattery(20); // only if it's electric
```

All trains can be stored together in a List structure:

```
List<Train> fleet = List.of(  
    new Train(),  
    new DieselTrain(),  
    new BatteryTrain(80)  
);  
  
for (Train t : fleet) {  
    t.start(); // all trains in fleet are started, each using its own start()  
}
```

If operations are needed for electric-specific trains only, then this can be invoked as

```
if (t instanceof ElectricTrain e) {  
    e.chargeBattery(10);  
}
```

-
5. Depending on whether the Vehicle (the top-class) is implemented as an abstract class or an interface, different options will result.

If implemented as an abstract class:

- You can provide shared fields or code (e.g., speed, engineStatus)
-

-
- A class can only extend one abstract class (Java limitation)
 - It limits flexibility because your classes cannot inherit from any other class

If implemented as an interface (modern Java style):

- Java interfaces can have default methods (since Java 8), narrowing the gap
- It allows multiple inheritance of behaviour
- Vehicle is a role, not a concrete object, and therefore an interface fits better

If any conclusion can (should) be formed on this basis, then it would be to use an interface. In general, an abstract class should only be chosen if you need shared implementation and you are sure subclasses won't need another parent class.

-
6. An example of how to add more vehicle types is provided here. This also demonstrates the power of implementing using interfaces.

```
public class Bicycle implements Vehicle {
    private int speed;
    @Override public void start() { speed = 1; }
    @Override public void stop() { speed = 0; }
    @Override public void accelerate(int amt) { speed += amt; }
    @Override public void decelerate(int amt) { speed = Math.max(0, speed - amt); }
    @Override public int getSpeed() { return speed; }
}
```

Using something like this allows us to reuse many of the known concepts:

Collections:

```
List<Vehicle> traffic = List.of(new Train(), new Bicycle());
```

Polymorphic operations:

```
for (Vehicle v : traffic)
    v.start(); // works for all
```

Comparators:

```
traffic.sort(Comparator.comparing(Vehicle::getSpeed));
```

Streams:

```
traffic.stream()
    .map(Vehicle::getSpeed)
    .reduce(Integer::max);
```
